

1. RDD PARTITIONS & HDFS BLOCKS

2. `defaultParallelism` / `defaultMinPartitions` behavior
 3. `countByValue` vs `reduceByKey`
-

1. RDD Partitions = HDFS Blocks (Mostly True)

What your notes say

- File = **1 GB (1024 MB)**
- HDFS block size = **128 MB**
- Blocks = $1024 / 128 = 8$
- $\Rightarrow$ RDD partitions = **8**

Why this is true

When you load data using:

```
spark.sparkContext.textFile("hdfs://...")
```

Spark:

- Reads **block-by-block**
- Creates **1 partition per block**

So:

👉 **Partitions $\approx$ Blocks in HDFS**

⚠️ **But here's the real-world nuance**

This is **not always exact**:

- Compression (like .gz) $\rightarrow$ fewer partitions (not splittable)
 - Small files $\rightarrow$ may get grouped
 - Custom input formats $\rightarrow$ can change partitioning
-

Senior Insight 💡

Partition count = **parallelism level of your job**

More partitions $\Rightarrow$ more parallel tasks

Too many $\Rightarrow$ overhead

Too few $\Rightarrow$ underutilized cluster

2. Why small data still gives 2 partitions?

Your example:

```
words_rdd = spark.sparkContext.parallelize(words)
```

```
words_rdd.getNumPartitions() # returns 2
```

Even though data is tiny (~KBs)

Key Concept: defaultParallelism

Spark enforces a **minimum number of partitions** when creating RDDs.

Rule:

```
numPartitions = max(calculated_partitions, defaultParallelism)
```

What is defaultParallelism?

```
spark.sparkContext.defaultParallelism
```

It usually equals:

- Number of CPU cores in cluster (or local machine)

Example:

- 2 cores → defaultParallelism = 2

Another Property: defaultMinPartitions

Used internally for some operations.

Rule from your notes:

- If partitions < 2 → use default (2)
- Else → use calculated value

Final Behavior

Data Size	Expected Partitions	Actual
Small (<128MB)	1	2 (because defaultParallelism)
350MB	3	3
1GB	8	8

Senior Insight 💡

This explains a common confusion:

“Why is my small dataset still parallelized?”

Because Spark is designed for **distributed execution first**, not single-thread optimization.

3. getNumPartitions()

This is straightforward:

```
rdd.getNumPartitions()
```

👉 Returns number of partitions in the RDD

Important

It tells you:

- How many **tasks** will run in parallel for transformations
-

4. countByValue vs reduceByKey

This is **very important** in interviews and real systems.

● countByValue()

What it does:

```
rdd.countByValue()
```

Equivalent to:

map → reduce → collect to driver

Key Characteristics

- ✅ Simple
 - ❌ Returns result to driver (local machine)
 - ❌ Not scalable for large data
-

Example from your image:

```
mapped_rdd.countByValue()
```

Output:

```
{  
  'CLOSED': 7556,  
  'PENDING_PAYMENT': 15030,  
  ...  
}
```

reduceByKey()

What it does:

```
rdd.map(lambda x: (x, 1)).reduceByKey(lambda a, b: a + b)
```

Key Characteristics

-  Distributed computation
-  Scalable
-  Supports further transformations

Key Differences

Feature	countByValue	reduceByKey
Type	Action	Transformation
Runs on	Driver	Cluster
Scalability	 Limited	 High
Output	Local dictionary	Distributed RDD
Further processing	 No	 Yes

Critical Rule (Senior-Level)

 Use countByValue **only when**:

- Data is small
- Final result is needed immediately

 Use reduceByKey when:

- Data is large

- You want further processing
-

Real-world failure scenario ⚠️

huge_rdd.countByValue()

💥 Driver OOM (Out Of Memory)

5. Hidden Concept: Why reduceByKey is better

Because of shuffle optimization

reduceByKey:

- Performs **local aggregation before shuffle**
- Reduces data movement

This is called:

👉 **Combiners optimization**

6. Putting it all together (Mental Model)

When reading data:

HDFS File → Blocks → Partitions → Tasks

When processing:

Partitions → Parallel Tasks → Transformations → Action

7. Practical Tips (From Industry)

✅ Control partitions manually

rdd.repartition(10)

rdd.coalesce(2)

✅ Ideal partition rule

Partitions $\approx 2-4 \times$ number of CPU cores

✅ Avoid this mistake

countByValue() on large datasets ❌

collect() on large datasets ❌

✅ **Prefer this pattern**

```
rdd.map(...).reduceByKey(...).saveAsTextFile()
```

8. Final Summary

- RDD partitions are usually based on **HDFS block size**
 - Spark enforces minimum partitions using **defaultParallelism**
 - Small data still gets parallelized
 - `countByValue` is **driver-heavy (dangerous for big data)**
 - `reduceByKey` is **distributed and scalable**
-

🧠 Big Picture First

Everything you're asking revolves around one core idea:

Parallelism in Spark = number of partitions

Because:

👉 **1 partition = 1 task = runs on 1 core**

So if you understand partitions, you understand:

- Performance
 - Resource utilization
 - Data distribution
-

💠 1. `getNumPartitions()`

What it does:

```
rdd.getNumPartitions()
```

👉 Returns **how many partitions your RDD currently has**

Why it matters:

- This tells you **how many parallel tasks will run**
 - Directly tied to **cluster utilization**
-

Example:

```
rdd = spark.sparkContext.parallelize([1,2,3,4], 2)
```

```
print(rdd.getNumPartitions()) # 2
```

👉 Means:

- 2 partitions
 - 2 tasks
 - Can use **2 CPU cores in parallel**
-

🔥 Interview Insight:

“getNumPartitions helps diagnose under-parallelism or excessive partitioning.”

♦ 2. defaultParallelism

What it is:

```
spark.sparkContext.defaultParallelism
```

👉 Default number of partitions Spark uses when **you don't specify**

How Spark decides this:

Depends on cluster:

Environment defaultParallelism

Local mode number of CPU cores

Cluster total cores across executors

Example:

If cluster has:

- 5 executors

- 4 cores each

👉 defaultParallelism = 20

🔥 Important:

This is used in:

```
spark.sparkContext.parallelize(data)
```

if you don't pass partition count.

Interview Answer:

“defaultParallelism reflects total available compute capacity and is used to decide initial task parallelism.”

◆ 3. defaultMinPartitions

Less commonly discussed — but important.

What it is:

Used in operations like:

```
sc.textFile("file.txt")
```

Formula (rough idea):

$$\text{minPartitions} = \max(\text{defaultParallelism}, 2)$$

What it means:

- Minimum number of partitions when reading data
 - Spark may create **more partitions based on file size**
-

Interview Insight:

“defaultMinPartitions ensures a baseline parallelism even for small inputs.”

🔥 Now the MOST IMPORTANT PART

📁 How a File is Split Across Partitions

This is what interviewers really care about.

◆ Case 1: `parallelize()` (your slides)

```
sc.parallelize(data, numPartitions)
```

How it works:

- Spark splits collection into chunks
- Each chunk → 1 partition

Example:

```
data = [1,2,3,4,5,6]
```

```
parallelize(data, 3)
```

👉 Partitions:

Partition Data

P1 [1,2]

P2 [3,4]

P3 [5,6]

◆ Case 2: Reading a File (`textFile`) — REAL WORLD

```
sc.textFile("hdfs://file")
```

Internally:

Spark uses **HDFS block size**

📦 HDFS Block Concept

Typical:

- 1 block = **128 MB**

Example: 1 GB file

1 GB / 128 MB = 8 blocks

👉 Spark creates:

8 partitions (default)

Mapping:

Block Partition

Block 1 Partition 1

Block 2 Partition 2

... ..

Key Insight:

“Spark creates one partition per HDFS block (by default)”

How Parallelism Affects This

You can override:

```
sc.textFile("file", minPartitions=20)
```

What happens?

- Spark **splits blocks further**
 - Creates more partitions than blocks
-

But:

 Spark **cannot combine blocks easily without shuffle**

Important Internal Behavior

1. Narrow Transformations

map(), filter()

- Partition count stays SAME
 - No reshuffling
-

2. Wide Transformations

reduceByKey(), join()

- Causes shuffle
- New partitioning created

Real Example End-to-End

Step 1: Read file

```
rdd = sc.textFile("file") # 8 partitions
```

Step 2: Transform

```
rdd.map(...)
```

👉 Still 8 partitions

Step 3: Reduce

```
rdd.reduceByKey(...)
```

👉 Shuffle happens

👉 New partitions created (defaultParallelism)

Critical Insight (Interview Gold)

“Input partitioning comes from storage (HDFS blocks), but after shuffle, partitioning is controlled by Spark’s parallelism settings.”

Common Performance Problems

Too few partitions

- CPUs idle
 - Slow job
-

Too many partitions

- Task overhead
 - Scheduler bottleneck
-

Ideal Partition Size

Rule of thumb:

100 MB – 200 MB per partition

Mental Model

File (HDFS)



Split into blocks (128MB)



Each block → Partition



Each partition → Task



Each task → Runs on 1 core

Final Interview-Level Summary

If asked:

“How does Spark decide partitions?”

You say:

“Initial partitioning is based on input data source, typically one partition per HDFS block. After shuffle operations, partitioning is controlled by Spark’s defaultParallelism or explicitly set values. Efficient parallelism requires balancing partition size and cluster resources.”

INTERVIEW-FOCUSED QUESTIONS ON THESE TOPICS:

1. Why does a small RDD still have multiple partitions?

Expected Answer (basic)

Because of defaultParallelism

✅ **Strong Answer (what interviewers want)**

Even if the dataset is small, Spark uses:

`spark.sparkContext.defaultParallelism`

to decide the **minimum number of partitions**.

So:



`partitions = max(calculated_partitions, defaultParallelism)`

💡 **Pro Insight**

- In local mode → number of CPU cores
 - In cluster → total executor cores
-

🔥 **2. What determines number of partitions in Spark?**

Answer Structure

There are **3 major cases**:

📌 **Case 1: Reading from HDFS**

👉 Partitions ≈ HDFS blocks

$\text{Partitions} = \text{File Size} / \text{Block Size}$

📌 **Case 2: `parallelize()`**

👉 Controlled by:

- `defaultParallelism`
- Optional parameter

`sc.parallelize(data, numSlices=10)`

📌 **Case 3: Transformations**

Some operations change partitions:

- `repartition(n)` → increases/decreases with shuffle
- `coalesce(n)` → reduces without shuffle

 Pro Insight

Partitioning is **not static** — it evolves through transformations.

 3. Difference between repartition() and coalesce()

Basic Answer

Feature	repartition	coalesce
Shuffle	Yes	No (mostly)
Increase partitions	Yes	No
Decrease partitions	Yes	Yes
Performance	Costly	Efficient

 Strong Explanation

- repartition():
 - Always shuffles
 - Ensures **even data distribution**
 - coalesce():
 - Avoids shuffle
 - Just merges partitions → can cause **data skew**
-

 When to use

- After filtering → use **coalesce**
 - Before heavy parallel processing → use **repartition**
-

 4. Why is countByValue() dangerous?

Basic Answer

Because it brings data to the driver

 Strong Answer

countByValue():

- Performs computation on cluster
- But **collects entire result to driver**

👉 This creates:

- Memory bottleneck
- Possible **driver OOM**

💡 **Better alternative**

```
rdd.map(lambda x: (x,1)).reduceByKey(lambda a,b: a+b)
```

🚨 **Interview Trap**

They may ask:

“When is countByValue okay?”

✅ Answer:

- Only when result size is small and final

🔥 **5. What is the difference between transformation and action?**

Answer

Type	Example	Execution
Transformation	map, reduceByKey	Lazy
Action	count, collect, countByValue	Triggers execution

💡 **Pro Insight**

Spark builds a **DAG (Directed Acyclic Graph)** and executes only when an **action** is called.

🔥 **6. What is a shuffle and why is it expensive?**

Answer

Shuffle = Data movement across partitions/nodes

Happens in:

- reduceByKey

- groupByKey
 - repartition
-

Why expensive?

- Disk I/O
 - Network transfer
 - Serialization/deserialization
-

💡 Senior Insight

Shuffle is the **most expensive operation in Spark**

🔥 7. Why is reduceByKey better than groupByKey?

Basic Answer

Because it reduces data before shuffle

✅ Strong Answer

- reduceByKey:
 - Performs **local aggregation (combiner)**
 - Sends less data across network
 - groupByKey:
 - Sends all values → heavy shuffle
-

💡 Golden Rule

Always prefer reduceByKey over groupByKey

🔥 8. How do you decide optimal number of partitions?

Standard Rule

Partitions = 2 to 4 × total CPU cores

Example

- 8 cores → 16–32 partitions

Advanced Insight

Too few partitions:

- CPU underutilized

Too many:

- Scheduling overhead
-

9. What happens if partitions are too large?

- Slow processing
 - Less parallelism
 - Memory pressure
-

10. What happens if partitions are too small?

- Too many tasks
 - High overhead
 - Scheduler bottleneck
-

Final Interview Tip

If interviewer asks anything about Spark performance:

 Always mention:

- Partitioning
 - Shuffle
 - Data locality
 - Serialization
-